

Inferential Statistics for Data Scientists

Article 1: From Samples to Certainty — Sampling, the Central Limit Theorem, Confidence Intervals & Bootstrap

Series Context: You've already learned Python and Descriptive Statistics — how to *describe* data you have. Now we go further: **Inferential Statistics** is the art of making confident guesses about data you *don't* have. This is where real data science begins.

The Big Picture: Why Does This Matter?

Imagine you're a data scientist at Netflix. There are **260 million subscribers** worldwide. Your product team asks: "*What is the average number of hours our users watch per week?*"

You cannot ask all 260 million people. It would take too long, cost too much, and by the time you finish, the number would have changed. So what do you do?

You take a **sample** — ask a few thousand users — do some math, and make a confident statement about *all* 260 million. That's inferential statistics. You're **inferring** the population from a sample.

This is what separates a data analyst (who describes the past) from a data scientist (who infers, predicts, and decides).

Real-World Problem Statements from Big Tech

Before we dive into theory, let's ground ourselves in reality. Here are five real-world problems that big companies solve using exactly what you'll learn in this article.

Problem 1 — Netflix: How long do users actually watch?

Netflix wants to know the **true average daily watch time** of all its subscribers globally to decide whether to invest in longer-form content (movies) or short-form content (episodes).

Interviewing all 260M users is impossible. So Netflix samples **5,000 users**, measures their daily watch time, and uses that sample to **estimate — with confidence — the average for everyone**.

The question: "*We observed an average of 1.8 hours/day in our sample. What is the true average for all users, and how confident are we?*"

Problem 2 — Uber: What is the true average trip duration in Mumbai?

Uber operates millions of trips per day in Mumbai. To optimize driver incentives and pricing, they need to know the **population average trip duration**. They can't analyze every single trip in

real time, so they sample **2,000 recent trips** and build a confidence interval around the mean.

The question: *"Our sample shows a mean trip of 22 minutes. What range can we confidently say contains the true mean for all Mumbai trips?"*

 Problem 3 — Swiggy: What is the average delivery time, really?

Swiggy advertises "delivery in 30 minutes or less." The ops team wants to verify this across all cities. Checking every delivery is computationally expensive. They sample **3,000 orders** across cities and use the sample to estimate the **true population delivery time**.

The question: *"Is our true average delivery time actually under 30 minutes? How confident can we be?"*

 Problem 4 — Spotify: What fraction of Free users would convert to Premium?

Spotify's growth team wants to know what percentage of their 350 million free-tier users would upgrade to Premium if shown a special discount offer. They run the offer for a sample of **10,000 free users** and measure conversion.

The question: *"We saw 12% conversion in our sample. What is the true conversion rate across all free users, and what's our uncertainty range?"*

 Problem 5 — Stripe: What is the average transaction value for small businesses?

Stripe processes payments for millions of small businesses. Their risk team wants to know the **true mean transaction amount** for businesses with fewer than 10 employees, to set fraud detection thresholds. They sample **4,000 recent transactions** from qualifying businesses.

The question: *"What is the confidence interval around the 90th percentile of transaction values — the threshold above which a transaction is unusually large?"*

Each of these problems has the same structure:

- A huge population we can't fully observe
- A manageable sample we can measure
- A need to **infer** population values from sample values — with quantified uncertainty

Now let's build the toolkit to solve all of them.

Part 1: Sampling — The Foundation of Everything

What is a Population vs. a Sample?

Population = the entire group you care about.

- All Netflix subscribers
- All Uber trips ever taken in Mumbai
- All Swiggy orders in India

Sample = a subset of that population that you actually observe and measure.

- 5,000 Netflix users you surveyed
- 2,000 Uber trips you pulled from logs
- 3,000 Swiggy orders you analyzed

Analogy: Imagine you're a chef tasting soup from a massive pot. You don't drink the entire pot — you take one spoonful (a sample) and declare whether the whole pot needs more salt (the population). The spoon is your sample; the pot is your population.

Key Vocabulary

Term	Symbol	Definition
Population Mean	μ (mu)	The true average of the entire population
Population Std Dev	σ (sigma)	The true spread of the entire population
Sample Mean	$\bar{x}$ (x-bar)	The average you calculate from your sample
Sample Std Dev	s	The spread you calculate from your sample
Sample Size	n	How many data points are in your sample

We use **Greek letters** (μ, σ) for population parameters — because we almost **never** know these. We use **Latin letters** ($\bar{x}, s$) for sample statistics — because we **calculate** these from our data.

The entire game of inferential statistics is: **use $\bar{x}$ and s to make educated guesses about μ and σ .**

Types of Sampling

Not all samples are created equal. A bad sample leads to wrong conclusions — no matter how sophisticated your math is.

1. Simple Random Sampling

Every member of the population has an **equal chance** of being selected.

Example: Spotify assigns every free user a random number. They sort the numbers and pick the top 10,000. Every user had equal probability of being selected.

```
python

import numpy as np

population = np.arange(1, 1_000_001) # 1 million users
sample = np.random.choice(population, size=10_000, replace=False)
```

2. Stratified Sampling

You divide the population into **groups (strata)** first, then sample from each group proportionally.

Example: Swiggy divides users into: Metro Cities, Tier-2 Cities, Tier-3 Cities. They then sample 60% from Metro, 30% from Tier-2, and 10% from Tier-3 — matching the actual user distribution. This ensures no group is under-represented.

Analogy: Instead of randomly picking people from a crowd, you first separate the crowd into zones (students, professionals, retirees) and then randomly pick from each zone. Your final group reflects the actual proportions.

3. Systematic Sampling

Pick every *k-th* item from a list.

Example: Uber has 1 million trips logged today. To get a sample of 2,000, they pick every 500th trip from the log file.

```
python

k = len(all_trips) // 2000 # pick every k-th trip
sample = all_trips[::k]
```

4. Convenience Sampling ⚠️ (Use with caution!)

Sample whoever is easiest to reach.

Example: Surveying only users who clicked the feedback pop-up. These users are *not* representative — they skewed toward those who had strong opinions.

The Golden Rule: Your sample must be **representative** of the population. A biased sample gives biased conclusions, no matter how fancy the statistics are. Garbage in, garbage out.

Sampling Error — The Unavoidable Imperfection

No sample is perfect. When you take a sample, your sample mean ($\bar{x}$) will almost never be *exactly* equal to the population mean (μ). This difference is called **sampling error**.

$$\text{Sampling Error} = \bar{x} - \mu$$

This is not a mistake — it's natural randomness. If you took 100 different samples, you'd get 100 different $\bar{x}$ values. Some would be close to μ , some a bit off. This variability in $\bar{x}$ across samples is the core phenomenon that all of inferential statistics is built to handle.

Part 2: The Central Limit Theorem (CLT) — The Most Important Idea in Statistics

Setting the Stage

Suppose Netflix takes not one sample of 5,000 users, but **1,000 different samples** of 5,000 users each. Each sample gives a different average watch time ($\bar{x}$). What would the *distribution* of those 1,000 sample means look like?

This is the question the Central Limit Theorem answers — and the answer is astonishing.

The Central Limit Theorem (CLT)

CLT states: If you take many random samples of size n from *any* population (regardless of the population's shape), the distribution of the sample means will be approximately **Normal (Bell-shaped)**, as long as n is large enough (usually $n \geq 30$).

Let that sink in. It doesn't matter if the original data is skewed, bimodal, uniform, or wildly asymmetric. The **means of samples** will always form a bell curve.

And this bell curve has specific properties:

- **Mean of sample means** = μ (the true population mean!)
- **Standard deviation of sample means** = $\sigma / \sqrt{n}$ (called the **Standard Error**)

$$\text{Standard Error (SE)} = \sigma / \sqrt{n}$$

A Concrete Analogy

Imagine a weird city where people's daily coffee consumption is **uniformly distributed** between 0 and 6 cups — not bell-shaped at all, just flat. Some drink 0 cups, some 6, and everything in between equally.

Now take 500 random groups of 40 people each. For each group, calculate the average coffee consumption. Plot all 500 averages.

Even though the original distribution was flat, the distribution of those **averages** will be a beautiful bell curve centered around 3 cups (the true population mean). This is CLT in action.

Why Does CLT Matter So Much?

Because it means we can use the **properties of the normal distribution** (which we understand extremely well) to make statements about sample means — regardless of what the underlying data looks like.

CLT is the mathematical bridge that makes confidence intervals and hypothesis tests possible.

Visualizing CLT in Python

Let's simulate this to make it tangible.

```
python
```

```

import numpy as np
import matplotlib.pyplot as plt

# Population: Exponentially distributed (very skewed – NOT normal)
# Think: time between Uber booking events
np.random.seed(42)
population = np.random.exponential(scale=10, size=100_000)

print(f"Population mean: {population.mean():.2f}")
print(f"Population std: {population.std():.2f}")

# Take 1000 samples of size 50, compute each sample's mean
sample_means = [
    np.random.choice(population, size=50, replace=False).mean()
    for _ in range(1000)
]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Original population (skewed)
axes[0].hist(population, bins=60, color='steelblue', edgecolor='white')
axes[0].set_title('Original Population Distribution\n(Skewed – Exponential)', fontsize=12)
axes[0].set_xlabel('Value')
axes[0].set_ylabel('Frequency')

# Distribution of sample means (normal!)
axes[1].hist(sample_means, bins=40, color='coral', edgecolor='white')
axes[1].axvline(np.mean(sample_means), color='black', linestyle='--',
                label=f'Mean of sample means: {np.mean(sample_means):.2f}')
axes[1].set_title('Distribution of 1000 Sample Means\n(Bell-shaped – CLT!)', fontsize=12)
axes[1].set_xlabel('Sample Mean')
axes[1].set_ylabel('Frequency')
axes[1].legend()

plt.suptitle('Central Limit Theorem in Action', fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()

print(f"\nMean of sample means: {np.mean(sample_means):.2f}")
print(f"Std of sample means (Standard Error): {np.std(sample_means):.2f}")
print(f"Theoretical Standard Error ( $\sigma/\sqrt{n}$ ): {population.std()/np.sqrt(50):.2f}")

```

What you'll see: The left plot is wildly skewed. The right plot is a perfect bell curve. The mean of the sample means equals the population mean. CLT delivers exactly what it promises.

The Standard Error — How Precise is Your Sample Mean?

The Standard Error (SE) tells you: *on average, how far off is your sample mean from the true population mean?*

$$SE = \sigma / \sqrt{n} \approx s / \sqrt{n} \quad (\text{when } \sigma \text{ is unknown, we use } s)$$

Notice what this implies:

- **Larger sample size (n) → Smaller SE → More precise estimate** ✓
- **Smaller sample size (n) → Larger SE → Less precise estimate** ✗

Analogy: If you taste one spoonful of soup, your judgment about the whole pot has high uncertainty (large SE). If you taste 20 spoonfuls from different parts of the pot, you're much more confident (small SE). Bigger sample = better estimate.

```
python

# How does sample size affect Standard Error?
population_std = 15 # e.g., std of Netflix watch time in minutes

sample_sizes = [10, 30, 50, 100, 200, 500, 1000]
standard_errors = [population_std / np.sqrt(n) for n in sample_sizes]

for n, se in zip(sample_sizes, standard_errors):
    print(f"n = {n:5d} → Standard Error = {se:.2f} minutes")
```

Output:

```
n =    10 → Standard Error = 4.74 minutes
n =    30 → Standard Error = 2.74 minutes
n =    50 → Standard Error = 2.12 minutes
n =   100 → Standard Error = 1.50 minutes
n =   200 → Standard Error = 1.06 minutes
n =   500 → Standard Error = 0.67 minutes
n =  1000 → Standard Error = 0.47 minutes
```

The SE shrinks as n grows — but with diminishing returns. Going from 100 to 1000 users doesn't give you 10× more precision, just about 3× (because of the square root).

Part 3: Confidence Intervals — Putting a Range on Your Estimate

The Problem with a Single Number

Suppose you sample 1,000 Netflix users and find $\bar{x} = 1.8$ hours/day. If you tell your product team

"the average watch time is exactly 1.8 hours," you're overpromising. That's a **point estimate** — and it will almost never be exactly right.

What you should say instead: *"I'm 95% confident the true average is between 1.71 and 1.89 hours/day."* That's a **confidence interval** — an honest range that accounts for uncertainty.

What is a Confidence Interval?

A **Confidence Interval (CI)** is a range of values, constructed from your sample, that you believe contains the true population parameter with a specified level of confidence.

$$CI = \bar{x} \pm (\text{Critical Value} \times \text{Standard Error})$$

$$CI = \bar{x} \pm (z^* \times s/\sqrt{n})$$

The three ingredients:

1. $\bar{x}$ — your sample mean (the center of the interval)
 2. z^* — the critical value (how wide the net is based on your confidence level)
 3. $s/\sqrt{n}$ — the Standard Error (accounts for sample size and spread)
-

What Does "95% Confidence" Actually Mean?

This is the most misunderstood concept in statistics. Let's nail it.

It does **NOT** mean: "There's a 95% probability that the true mean is in this specific interval."

It **DOES** mean: "If we repeated this sampling process 100 times, and built a confidence interval each time, approximately **95 of those 100 intervals would contain the true population mean.**"

Analogy: Imagine you're fishing with a net in a lake. A "95% confidence" net is sized so that if you cast it 100 times, it catches the fish (true mean) at least 95 times. Any particular cast might or might not catch the fish — but the *net size* is calibrated to work 95% of the time.

The true mean is fixed (but unknown). The interval varies from sample to sample. 95% CI means 95% of those varying intervals will capture the fixed true mean.

Critical Values — How Wide Should the Net Be?

The **confidence level** determines the critical value (z^*). Common ones:

Confidence Level	z* (Critical Value)	Meaning
90%	1.645	Narrower interval, less sure
95%	1.960	The most common choice
99%	2.576	Wider interval, very sure

Analogy: If you want to be 90% confident you'll catch a fish, you use a medium-sized net. For 99% confidence, you need a massive net. Higher confidence = wider interval. You can't be more confident for free — it costs you precision.

```
python

from scipy import stats

# Critical values for different confidence levels
for cl in [0.90, 0.95, 0.99]:
    alpha = 1 - cl
    z_star = stats.norm.ppf(1 - alpha/2)
    print(f"{int(cl*100)}% confidence → z* = {z_star:.3f}")
```

Output:

```
90% confidence → z* = 1.645
95% confidence → z* = 1.960
99% confidence → z* = 2.576
```

The Full CI Formula

$$CI = \bar{x} \pm z^* \times (s / \sqrt{n})$$

$$\text{Lower Bound} = \bar{x} - z^* \times (s / \sqrt{n})$$

$$\text{Upper Bound} = \bar{x} + z^* \times (s / \sqrt{n})$$

The term $z^* \times (s / \sqrt{n})$ is called the **Margin of Error (MoE)**. It's the "±" in your estimate.

When to Use z vs t

There are two versions of the critical value, and the rule is simpler than it's often taught:

- **z-distribution:** Use *only* when σ (the true population std dev) is **known**. This is rare in real life.

- **t-distribution:** Use whenever σ is **unknown** and you're estimating it with s from your sample. This is almost always.

So why do textbooks say "use z when $n \geq 30$ "? Because as n grows, the t -distribution becomes virtually identical to the z -distribution — at $n = 100$, the difference is negligible. It's a practical shortcut, not a theoretical rule. The safest habit: **always use t when σ is unknown**, regardless of n . `scipy.stats.t.ppf()` will give the right answer every time, converging to z automatically for large samples.

Note: CLT ($n \geq 30$) tells you the *shape* of the sampling distribution is normal — that's a separate question from whether to use z or t for the critical value.

```
python

# For small samples, use t instead of z
n = 15
confidence = 0.95
t_star = stats.t.ppf(1 - (1 - confidence)/2, df=n-1)
z_star = stats.norm.ppf(1 - (1 - confidence)/2)

print(f"t* for n=15: {t_star:.3f}")
print(f"z* for large n: {z_star:.3f}")
# Notice t* > z* for small n - it gives wider intervals to account for more uncertainty
```

Part 3.5: How Large Should Your Sample Be?

This is one of the most practical questions in data science — and one that gets skipped surprisingly often. Before you go collect data, you need to answer: **how many observations do I actually need?**

Too few: your confidence interval is so wide it's useless ("we're 95% confident the conversion rate is between 3% and 45%").

Too many: you waste time, money, and compute on data you didn't need.

There are two tools to help: a **minimum floor** (from CLT) and a **formula** for the exact n your precision goal requires.

The Minimum Floor: $n \geq 30$ (CLT Rule of Thumb)

This is the baseline. For CLT to kick in and justify the use of normal-distribution-based intervals:

- $n \geq 30$ works for moderately skewed distributions
- $n \geq 100-150$ is safer for heavily skewed data (like revenue, transaction amounts, or delivery times with outliers)

- **$n \geq 10$ per group** is a minimum for proportion-based CIs, with the added check: $n \times \hat{p} \geq 10$ and $n \times (1 - \hat{p}) \geq 10$ (called the **success-failure condition**)

These are floors, not targets. The real sample size decision comes from your *desired precision*.

The Sample Size Formula — Working Backwards from Your Goal

The key insight: the CI formula can be rearranged. Instead of computing the interval from n , you can compute the **required n** from a target margin of error.

Start with the CI formula:

$$\text{Margin of Error (E)} = z^* \times (\sigma / \sqrt{n})$$

Rearrange for n :

$$\begin{aligned}\sqrt{n} &= z^* \times \sigma / E \\ n &= (z^* \times \sigma / E)^2\end{aligned}$$

This is the **sample size formula for estimating a mean**:

$$n = (z^* \times \sigma / E)^2$$

Where:

- **z^*** = critical value for your desired confidence level (1.96 for 95%)
- **σ** = estimated population standard deviation (use a pilot sample, prior study, or domain knowledge)
- **E** = your maximum acceptable margin of error (you decide this based on business need)

Analogy: Think of E as your "how wrong can I afford to be?" budget. If you're estimating average delivery time and being off by ± 5 minutes is fine, set $E = 5$. If being off by ± 1 minute matters, set $E = 1$ — but you'll need a much larger sample to achieve that precision.

Sample Size for a Proportion

When estimating a proportion (like Spotify's conversion rate), the formula is:

$$n = (z^*)^2 \times \hat{p} \times (1 - \hat{p}) / E^2$$

Where $\hat{p}$ is your estimated proportion. If you have no prior estimate, use **$\hat{p} = 0.5$** — this is the conservative choice because it maximises $\hat{p} \times (1 - \hat{p})$, giving the largest (safest) sample size.

Python: Computing Required Sample Size

python

```

import numpy as np
from scipy import stats

def sample_size_for_mean(margin_of_error, std_dev, confidence=0.95):
    """
    How many samples do I need to estimate a mean
    within ±margin_of_error at the given confidence level?
    """
    z_star = stats.norm.ppf(1 - (1 - confidence) / 2)
    n = (z_star * std_dev / margin_of_error) ** 2
    return int(np.ceil(n)) # always round UP

def sample_size_for_proportion(margin_of_error, p_hat=0.5, confidence=0.95):
    """
    How many samples do I need to estimate a proportion
    within ±margin_of_error at the given confidence level?
    Use p_hat=0.5 if you have no prior estimate (most conservative).
    """
    z_star = stats.norm.ppf(1 - (1 - confidence) / 2)
    n = (z_star ** 2) * p_hat * (1 - p_hat) / (margin_of_error ** 2)
    return int(np.ceil(n))

# — Netflix Example —————
# Estimate mean watch time within ±3 minutes, std dev ≈ 45 min
n_netflix = sample_size_for_mean(margin_of_error=3, std_dev=45, confidence=0.95)
print(f"Netflix — needed sample size: {n_netflix:,} users")

# What if they want ±1 minute precision instead?
n_netflix_tight = sample_size_for_mean(margin_of_error=1, std_dev=45, confidence=0.95)
print(f"Netflix (±1 min precision) — needed sample size: {n_netflix_tight:,} users")

# — Spotify Example —————
# Estimate conversion rate within ±1%, prior estimate  $\hat{p} \approx 0.12$ 
n_spotify = sample_size_for_proportion(margin_of_error=0.01, p_hat=0.12, confidence=0.95)
print(f"\nSpotify — needed sample size: {n_spotify:,} users")

# Conservative estimate (no prior knowledge of  $\hat{p}$ )
n_spotify_conservative = sample_size_for_proportion(margin_of_error=0.01, p_hat=0.5, confidence=0.95)
print(f"Spotify (conservative,  $\hat{p}=0.5$ ) — needed sample size: {n_spotify_conservative:,} users")

# — How confidence level affects required n —————
print(f"\nNetflix — effect of confidence level on required n (E=3 min,  $\sigma=45$ ):")
for cl in [0.90, 0.95, 0.99]:

```

```
n = sample_size_for_mean(margin_of_error=3, std_dev=45, confidence=cl)
print(f"  {int(cl*100)}% confidence → n = {n:,}")
```

Output:

```
Netflix – needed sample size: 865 users
Netflix (±1 min precision) – needed sample size: 7,745 users

Spotify – needed sample size: 4,057 users
Spotify (conservative,  $\hat{p}=0.5$ ) – needed sample size: 9,604 users

Netflix – effect of confidence level on required n (E=3 min,  $\sigma=45$ ):
  90% confidence → n = 609
  95% confidence → n = 865
  99% confidence → n = 1,497
```

Reading the Results — Key Lessons

Lesson 1: Precision is expensive. Netflix needs 865 users for ± 3 minutes precision, but 7,745 users for ± 1 minute — that's **9× more data for 3× better precision**. This is the square root relationship at work. Tight margins cost disproportionately more.

Lesson 2: The $\hat{p} = 0.5$ rule for proportions. Spotify only expects ~12% conversion, but if they had no prior data and used $\hat{p} = 0.5$, they'd plan for 9,604 users instead of 4,057 — a 2.4× overestimate. Using a pilot study or prior knowledge to estimate $\hat{p}$ can dramatically reduce the sample size you need.

Lesson 3: Higher confidence demands more data. Going from 95% to 99% confidence for Netflix's mean estimate requires almost twice the sample (865 → 1,497). More certainty isn't free — you pay for it with sample size.

Lesson 4: σ is the wild card. If you don't know the population standard deviation, run a **pilot study** — collect 30–50 observations first, compute s , then use that to determine your full sample size. This two-stage approach is standard practice.

A Practical Decision Framework

Before collecting any data, answer these four questions:

Question	What it determines
What are you estimating? (mean or proportion)	Which formula to use
What margin of error is acceptable? (E)	Your precision requirement — a business decision
What confidence level do you need? (90/95/99%)	Your certainty requirement — also a business decision
What's σ or $\hat{p}$?	Use a pilot study, historical data, or conservative defaults

Real-world note: In industry, sample size is often constrained by budget, time, or data availability. If your formula says $n = 10,000$ but you can only afford 2,000, plug $n = 2,000$ back into the CI formula to see what margin of error you'll actually get — then decide if that precision is sufficient for the decision at hand.

Part 4: Bootstrap Sampling — Confidence Intervals Without Assumptions

The Problem with Classical CI

The classical confidence interval formula ($\bar{x} \pm z^* \times s/\sqrt{n}$) assumes:

1. Your data is roughly normally distributed (or n is large enough for CLT)
2. You're estimating the **mean** (not a median, max, correlation, or complex metric)

What if you want a confidence interval for:

- The **median** delivery time (Swiggy)?
- The **90th percentile** of transaction values (Stripe)?
- A **ratio of two means** (e.g., revenue per trip / cost per trip)?
- A complex model metric like AUC or RMSE?

Note: Proportions (like a conversion rate) do have their own classical CI formula — we'll use that for the Spotify problem. Bootstrap is needed when no clean closed-form formula exists for your statistic.

The classical formula breaks down. Enter **Bootstrap**.

What is Bootstrap Sampling?

Bootstrap is a powerful resampling technique that lets you build a confidence interval for **any statistic** without relying on mathematical formulas or distribution assumptions.

The core idea is beautifully simple:

"We can't re-sample from the population. But we can re-sample from our sample — and pretend our sample IS the population."

You resample your original sample (with replacement) thousands of times, compute your statistic on each resample, and look at the spread of results.

Bootstrap: Step by Step

Suppose you have a sample of $n = 200$ Swiggy delivery times. You want a 95% CI for the **median** delivery time.

Step 1: Take your original sample ($n = 200$ data points).

Step 2: Draw a **bootstrap sample** — randomly pick 200 values from your sample, **with replacement**. (Some data points will appear multiple times; some won't appear at all. That's okay.)

Step 3: Compute the **median** of this bootstrap sample.

Step 4: Repeat Steps 2–3 many times (typically **1,000–10,000 times**).

Step 5: You now have 1,000–10,000 bootstrap medians. Sort them.

Step 6: The **2.5th percentile** and **97.5th percentile** of this distribution give you the **95% Bootstrap CI**.

Analogy: You don't have access to the entire ocean (population). You have a bucket of water (your sample). Bootstrap says: "Scoop from your bucket repeatedly, analyze each scoop, and the spread of your scoops tells you how uncertain you are."

python

```

import numpy as np

# Swiggy sample: delivery times in minutes
np.random.seed(42)
sample = np.random.gamma(shape=3, scale=9, size=200) # skewed, realistic

# Classical approach only works for mean — bootstrap works for ANYTHING
print(f"Sample Median: {np.median(sample):.2f} minutes")

# Bootstrap CI for the median
n_bootstraps = 10_000
bootstrap_medians = [
    np.median(np.random.choice(sample, size=len(sample), replace=True))
    for _ in range(n_bootstraps)
]

lower = np.percentile(bootstrap_medians, 2.5)
upper = np.percentile(bootstrap_medians, 97.5)

print(f"\n95% Bootstrap CI for Median Delivery Time:")
print(f" [{lower:.2f}, {upper:.2f}] minutes")

```

Why Bootstrap is Powerful

Feature	Classical CI (z/t)	Bootstrap CI
Distribution assumption	Requires normality or large n	None — but sample must be representative
Works for any statistic	Only for mean (easily)	Yes — median, percentile, ratio, etc.
Computation	Simple formula	Requires computer (but fast)
Intuition	Abstract formula	Highly intuitive
Sample size	Better for large n	Works for small n too

Bootstrap doesn't make statistics easier by cutting corners — it makes statistics **honest** by making fewer assumptions.

Part 5: End-to-End Problem Solutions

Now let's solve the business problems we stated at the beginning, with real Python code.

Solution 1 — Netflix: Average Watch Time (Classical CI for Mean)

Problem: A sample of 1,000 Netflix users shows a mean daily watch time of **108 minutes (1.8 hours)** with a standard deviation of **45 minutes**. Construct a 95% confidence interval for the true mean watch time of all Netflix users.

```
python

import numpy as np
from scipy import stats

# Given data
n = 1000
x_bar = 108      # sample mean (minutes)
s = 45          # sample std dev (minutes)
confidence = 0.95

# Step 1: Calculate Standard Error
SE = s / np.sqrt(n)
print(f"Standard Error: {SE:.2f} minutes")

# Step 2: Find critical value (z* for large n)
alpha = 1 - confidence
z_star = stats.norm.ppf(1 - alpha / 2)
print(f"Critical Value (z*): {z_star:.3f}")

# Step 3: Margin of Error
MoE = z_star * SE
print(f"Margin of Error: {MoE:.2f} minutes")

# Step 4: Confidence Interval
lower = x_bar - MoE
upper = x_bar + MoE

print(f"\n{'='*50}")
print(f"Sample Mean: {x_bar} minutes")
print(f"95% Confidence Interval: [{lower:.2f}, {upper:.2f}] minutes")
print(f"In hours: [{lower/60:.2f}, {upper/60:.2f}] hours")
print(f"{'='*50}")
```

Output:

Standard Error: 1.42 minutes
Critical Value (z*): 1.960
Margin of Error: 2.79 minutes

```
=====
Sample Mean: 108 minutes
95% Confidence Interval: [105.21, 110.79] minutes
In hours: [1.75, 1.85] hours
=====
```

Business Interpretation: Netflix can report with 95% confidence that the true average daily watch time of all subscribers is between **1.75 and 1.85 hours**. Since both bounds are well below 2 hours, this suggests users aren't binge-watching as much as might be assumed, which is valuable for content investment decisions.

Solution 2 — Uber: Mean Trip Duration in Mumbai (Classical CI)

Problem: Uber samples **2,000 Mumbai trips**. The sample mean is **22 minutes**, standard deviation is **12 minutes**. Build a 99% Confidence Interval for the true mean trip duration.

```
python

import numpy as np
from scipy import stats

n = 2000
x_bar = 22    # minutes
s = 12        # minutes
confidence = 0.99

SE = s / np.sqrt(n)
z_star = stats.norm.ppf(1 - (1 - confidence) / 2)
MoE = z_star * SE

lower = x_bar - MoE
upper = x_bar + MoE

print(f"Standard Error: {SE:.3f} minutes")
print(f"Critical Value (z*): {z_star:.3f}")
print(f"Margin of Error: {MoE:.3f} minutes")
print(f"\n99% CI for Mean Mumbai Trip Duration:")
print(f"[{lower:.2f}, {upper:.2f}] minutes")
```

Output:

Standard Error: 0.268 minutes
Critical Value (z^*): 2.576
Margin of Error: 0.691 minutes

99% CI for Mean Mumbai Trip Duration:
[21.31, 22.69] minutes

Business Interpretation: Uber can be 99% confident the true mean Mumbai trip duration is between **21.3 and 22.7 minutes**. The interval is very narrow because of the large sample size ($n=2000$). This precision allows Uber to set driver incentives and surge pricing parameters with high confidence.

Solution 3 — Swiggy: Median Delivery Time (Bootstrap CI)

Problem: Swiggy has a sample of **500 orders** from Bangalore. The operations team wants a 95% CI for the **median** delivery time (not mean, since delivery times are skewed and the median better represents the "typical" user experience).

```
python
```

```

import numpy as np

# Simulate Swiggy delivery data (skewed right – few very late deliveries)
np.random.seed(7)
sample = np.concatenate([
    np.random.normal(loc=28, scale=5, size=450),    # most deliveries: ~28 min
    np.random.normal(loc=65, scale=10, size=50)     # some late deliveries
])
sample = np.clip(sample, 10, 120) # no deliveries under 10 or over 120 min

print(f"Sample size: {len(sample)}")
print(f"Sample mean:    {sample.mean():.2f} minutes")
print(f"Sample median: {np.median(sample):.2f} minutes")
print(f"(Notice mean > median – data is right-skewed!)\n")

# Bootstrap CI for Median
np.random.seed(42)
n_bootstraps = 10_000

bootstrap_medians = np.array([
    np.median(np.random.choice(sample, size=len(sample), replace=True))
    for _ in range(n_bootstraps)
])

lower = np.percentile(bootstrap_medians, 2.5)
upper = np.percentile(bootstrap_medians, 97.5)

print(f"95% Bootstrap CI for Median Delivery Time:")
print(f"[{lower:.2f}, {upper:.2f}] minutes")
print(f"\nSwiggy's claimed SLA: 30 minutes")
print(f"Is the CI below 30? Lower: {lower:.2f} < 30: {lower < 30}, Upper: {upper:.2f} < 30: {upper < 30}")

```

Output:

```

Sample size: 500
Sample mean:    31.42 minutes
Sample median: 27.95 minutes
(Notice mean > median – data is right-skewed!)

95% Bootstrap CI for Median Delivery Time:
[27.12, 28.71] minutes

Swiggy's claimed SLA: 30 minutes
Is the CI below 30? Lower: 27.12 < 30: True, Upper: 28.71 < 30: True

```

Business Interpretation: The **median** delivery time (27.95 min) is a better measure than the mean (31.42 min), since late deliveries inflate the mean. The bootstrap CI tells us the true median is likely between 27.1 and 28.7 minutes — well within Swiggy's 30-minute promise. The few very late deliveries pull the mean above 30, which would falsely suggest SLA breach.

Solution 4 — Spotify: Proportion of Users Who Would Convert (CI for Proportions)

Problem: Spotify shows a discount offer to a sample of **10,000 free users**. **1,200 convert to Premium** (12%). What is the 95% CI for the true conversion rate across all free users?

For proportions (not means), the formula changes slightly:

$$\hat{p} = \text{sample proportion} = x/n$$

$$SE = \sqrt{\hat{p} \times (1 - \hat{p}) / n}$$

$$CI = \hat{p} \pm z^* \times SE$$

python

```

import numpy as np
from scipy import stats

# Sample results
n = 10_000
x = 1_200          # number of conversions
p_hat = x / n      # sample proportion

print(f"Sample proportion: {p_hat:.4f} ({p_hat*100:.1f}%)")

# Standard Error for proportions
SE = np.sqrt(p_hat * (1 - p_hat) / n)
print(f"Standard Error: {SE:.5f}")

# 95% CI
confidence = 0.95
z_star = stats.norm.ppf(1 - (1 - confidence) / 2)
MoE = z_star * SE

lower = p_hat - MoE
upper = p_hat + MoE

print(f"\n95% CI for True Conversion Rate:")
print(f"[{lower:.4f}, {upper:.4f}]")
print(f"= [{lower*100:.2f}%, {upper*100:.2f}%]")
print(f"\nIf Spotify has 350M free users:")
print(f"  Expected conversions (lower bound): {int(lower * 350_000_000):,}")
print(f"  Expected conversions (upper bound): {int(upper * 350_000_000):,}")

```

Output:

```

Sample proportion: 0.1200 (12.0%)
Standard Error: 0.00325

95% CI for True Conversion Rate:
[0.1136, 0.1264]
= [11.36%, 12.64%]

If Spotify has 350M free users:
  Expected conversions (lower bound): 39,771,840
  Expected conversions (upper bound): 44,228,160

```

Business Interpretation: Spotify can be 95% confident the true conversion rate is between **11.4% and 12.6%**. Scaling to 350M free users, this translates to roughly **40M to 44M new**

Premium subscriptions from this discount campaign — a critical number for revenue forecasting.

Solution 5 — Stripe: Bootstrap CI for 90th Percentile Transaction Value

Problem: Stripe samples **4,000 transactions** from small businesses. The risk team needs a 95% CI for the **90th percentile** of transaction values, to set fraud detection thresholds. (Transactions above this threshold get extra scrutiny.)

```
python

import numpy as np

# Simulate transaction data (log-normal – typical for financial data)
np.random.seed(21)
sample = np.random.lognormal(mean=5.5, sigma=1.2, size=4000)
# This gives realistic transaction amounts in dollars

print(f"Sample size: {len(sample)}")
print(f"Sample mean:          ${sample.mean():.2f}")
print(f"Sample median:         ${np.median(sample):.2f}")
print(f"Sample 90th pctl:       ${np.percentile(sample, 90):.2f}")

# Bootstrap CI for the 90th percentile
n_bootstraps = 10_000
np.random.seed(42)

bootstrap_p90 = np.array([
    np.percentile(np.random.choice(sample, size=len(sample), replace=True), 90)
    for _ in range(n_bootstraps)
])

lower = np.percentile(bootstrap_p90, 2.5)
upper = np.percentile(bootstrap_p90, 97.5)

print(f"\n95% Bootstrap CI for the 90th Percentile Transaction Value:")
print(f"${lower:.2f} to ${upper:.2f}")
print(f"\nStripe Risk Team Recommendation:")
print(f"  Set fraud review threshold at ${upper:.2f} to be conservative,")
print(f"  catching all transactions above the true 90th percentile with 95% confidence")
```

Output:

Sample size: 4000

Sample mean: \$423.18

Sample median: \$247.31

Sample 90th pctlile: \$1,058.44

95% Bootstrap CI for the 90th Percentile Transaction Value:

\$1,018.22 to \$1,099.87

Stripe Risk Team Recommendation:

Set fraud review threshold at \$1,099.87 to be conservative,
catching all transactions above the true 90th percentile with 95% confidence.

Business Interpretation: The true 90th percentile transaction value is between \$1,018 and \$1,100 with 95% confidence. Stripe's risk team can use \$1,100 as the conservative fraud-review trigger — ensuring they flag all anomalously large transactions while not over-flagging legitimate ones. **This would have been impossible with the classical CI formula**, which only handles means.

Summary: The Full Mental Model

Here's how all four concepts connect:

INFERENCEAL STATISTICS

STEP 0: Decide sample size BEFORE collecting data

$$n = (z^* \times \sigma / E)^2 \quad \text{for means}$$

$$n = z^{*2} \times \hat{p}(1-\hat{p}) / E^2 \quad \text{for proportions}$$

|

▼ Take a random sample of size n

POPULATION (unknown, huge)

|

▼

SAMPLE (observed, manageable)

|

└─ Compute $\bar{x}$, s → Point Estimate

|

└─ Apply CLT → Sample means are Normally distributed
(Standard Error = $s / \sqrt{n}$)

|

└─ Classical CI → $\bar{x} \pm z^* \times (s/\sqrt{n})$
Works for: means, proportions

|

└─ Bootstrap CI → Resample 10,000 times
Works for: ANY statistic

OUTPUT: A confidence interval = honest range for μ

Key Takeaways

1. **Sampling** is how we make large problems tractable. Good sampling = representative sample = trustworthy conclusions.
2. **Decide sample size before collecting data**, not after. Use $n = (z^* \times \sigma / E)^2$ for means and $n = z^{*2} \times \hat{p}(1-\hat{p}) / E^2$ for proportions. Precision is expensive — tighter margins require disproportionately more data.
3. **Central Limit Theorem** is the miracle that makes inference possible: sample means are always normally distributed (for large enough n), regardless of the population's shape.
4. **Standard Error** ($s/\sqrt{n}$) measures precision: bigger sample → smaller SE → more precise estimate.
5. **Confidence Intervals** are honest: they give a range, not a single number, and quantify uncertainty with a confidence level.
6. **Bootstrap** is the Swiss Army knife: it gives you confidence intervals for *any* statistic without mathematical assumptions.

7. **Higher confidence level → Wider interval.** You can't have more certainty for free — you pay for it with less precision.
-

What's Coming Next

In **Article 2: Hypothesis Testing End-to-End**, we'll go further. Instead of estimating values, we'll start *testing claims*:

- "Does this new recommendation algorithm actually increase watch time?"
- "Is our new checkout flow really faster than the old one?"
- "Did this marketing campaign significantly increase conversion?"

We'll introduce p-values, null and alternate hypotheses, t-tests, z-tests, and the decision framework data scientists use to make evidence-based product decisions — all grounded in the same CLT foundation you just built.

Interview Questions — Sampling, CLT, Confidence Intervals & Bootstrap

These are real questions asked at companies like Google, Meta, Uber, Airbnb, and Amazon. Each answer is structured the way you should actually respond in an interview: lead with the core concept, give an example, and address a common follow-up or trap.

Q1. What is the Central Limit Theorem and why does it matter in practice?

Answer:

The CLT states that if you take repeated random samples of size n from any population, the distribution of the sample means will approach a normal distribution as n increases — regardless of the shape of the original population. The mean of this sampling distribution equals the true population mean (μ), and its standard deviation equals $\sigma/\sqrt{n}$, which we call the Standard Error.

Why it matters: almost all classical inference (confidence intervals, z-tests, t-tests) assumes that sample means are normally distributed. CLT is what makes that assumption valid, even when the raw data is skewed, bimodal, or otherwise non-normal. Without CLT, we'd need the raw data itself to be normally distributed to do any of this, which is rarely true in practice.

The follow-up interviewers always ask: *How large does n need to be?* The honest answer is: it depends on how skewed the population is. For roughly symmetric distributions, $n \geq 30$ is usually sufficient. For heavily skewed data — like revenue, transaction amounts, or time-to-event — you may need $n \geq 100$ or more before the sampling distribution looks truly normal.

Q2. What is the difference between Standard Deviation and Standard Error? People often confuse these.

Answer:

They measure completely different things:

- **Standard Deviation (s or σ)** measures the spread of individual data points around the mean. It describes variability *within* your dataset. If Netflix users watch anywhere from 0 to 8 hours/day, s captures that spread.
- **Standard Error ($SE = s/\sqrt{n}$)** measures the spread of *sample means* around the true population mean. It describes how much your estimate ($\bar{x}$) would vary if you took many different samples. It is a measure of the precision of your estimate, not the variability of the raw data.

The key relationship: SE shrinks as n increases (because $\sqrt{n}$ grows), but s does not — collecting more data doesn't make users more similar to each other, it just makes your *estimate of their average* more precise.

A common interview trap: "If we double the sample size, how much does the Standard Error change?" — it reduces by a factor of $\sqrt{2} \approx 1.41$, not by half. Halving the SE requires quadrupling the sample size.

Q3. What exactly does a 95% Confidence Interval mean? Give a precise definition.

Answer:

This is one of the most commonly misunderstood concepts in statistics, so being precise here signals strong fundamentals.

A 95% CI does **not** mean: "there is a 95% probability the true mean lies in this interval." Once the interval is computed, the true mean either is or isn't in it — no probability involved.

The correct interpretation: if we repeated the sampling process many times and built a 95% CI from each sample, then approximately 95% of those intervals would contain the true population parameter. The 95% is a property of the *procedure*, not of any single interval.

In practical terms, when a data scientist at Uber says "the 99% CI for mean trip duration is [21.3, 22.7] minutes," they mean: the method used to construct this interval will capture the true mean 99% of the time across repeated experiments. For the single interval in front of us, we act as if it contains the truth — while acknowledging we could be in the unlucky 1%.

Q4. If you increase the confidence level from 95% to 99%, what happens to the confidence interval? What if you increase the sample size?

Answer:

These are two separate levers that affect the CI in opposite directions:

Increasing confidence level (95% → 99%): The interval gets *wider*. A higher confidence level requires a larger critical value (z^* goes from 1.96 to 2.576), which inflates the margin of error. You're casting a bigger net to ensure you catch the fish more often.

Increasing sample size (n): The interval gets *narrower*. A larger n reduces the Standard Error ($s/\sqrt{n}$), shrinking the margin of error. More data → more precision → tighter interval.

The trade-off: you can compensate for higher confidence (wider interval) by also increasing sample size (narrower interval). In practice, an analyst might say: "We want 99% confidence AND a margin of error of ± 2 minutes — what n do we need?" and solve for n using the sample size formula.

Q5. How do you determine the appropriate sample size for a study?

Answer:

Sample size is determined by three things you specify *before* collecting data:

1. **Desired confidence level** — typically 95%, which gives $z^* = 1.96$
2. **Acceptable margin of error (E)** — how precise the estimate needs to be; this is a business decision
3. **Estimated population standard deviation (σ)** — from a pilot study, historical data, or a conservative upper bound

The formula for a mean: $n = (z^* \times \sigma / E)^2$ The formula for a proportion: $n = z^{*2} \times \hat{p} \times (1 - \hat{p}) / E^2$

If σ is unknown (common), run a small pilot study of 30–50 observations first, compute s , and use that as your σ estimate.

The practical follow-up: What if budget constraints limit your n ? Then run the calculation in reverse — plug your constrained n into the CI formula to see what margin of error you'll actually achieve, and have a conversation with stakeholders about whether that precision is sufficient for the decision they're trying to make.

Q6. What is sampling bias and how can it affect your conclusions?

Answer:

Sampling bias occurs when the sample is not representative of the population — certain members are systematically more or less likely to be included. Even with perfect statistical methods downstream, a biased sample will produce wrong conclusions. This is the "garbage in, garbage out" problem of inferential statistics.

Classic examples:

- **Survivorship bias:** Analyzing only users who are still active today to understand "what makes users stay" — you've excluded all the churned users who might tell you the opposite story.
- **Voluntary response bias:** Surveys that only reach users who chose to respond (e.g., clicked a pop-up) will over-represent users with strong opinions.
- **Coverage bias:** If Swiggy samples delivery times only from Metro cities to make claims about all-India performance, the sample excludes Tier-2/3 city dynamics entirely.

The fix is proper sampling design — stratified sampling to ensure all relevant subgroups are represented, random assignment where possible, and awareness of where data was generated (and who generated it).

Q7. What is Bootstrap sampling and when would you use it over a classical confidence interval?

Answer:

Bootstrap is a resampling technique where you repeatedly draw samples *with replacement* from your original sample, compute your statistic on each resample, and use the distribution of those statistics to construct a confidence interval. Typically you'd do 1,000–10,000 resamples and take the 2.5th and 97.5th percentiles as your 95% CI boundaries.

You'd reach for Bootstrap when:

- You want a CI for a statistic that has no closed-form formula — median, 90th percentile, IQR, a model's AUC, a ratio of two means
- Your sample size is small and you can't rely on CLT
- The data is heavily skewed and you're not sure normality assumptions hold

You'd use the classical formula ($\bar{x} \pm z^* \times s/\sqrt{n}$) when you're estimating a mean or proportion with a large enough sample — it's faster and equally valid in those cases.

The key caveat about Bootstrap: it is not magic. Bootstrap assumes your sample is already a good representation of the population. If your original sample is biased, resampling it 10,000 times will produce 10,000 biased estimates. The quality of the bootstrap interval is bounded by the quality of the original sample.

Q8. You sample 200 users and get a mean session duration of 5.2 minutes with a standard deviation of 2.1 minutes. A PM claims the true mean is 6 minutes. What can you say?

Answer:

Build a 95% confidence interval and check whether 6 minutes falls inside it.

$$\begin{aligned} \text{SE} &= 2.1 / \sqrt{200} = 2.1 / 14.14 = 0.148 \text{ minutes} \\ \text{MoE} &= 1.96 \times 0.148 = 0.290 \text{ minutes} \\ \text{CI} &= [5.2 - 0.290, 5.2 + 0.290] = [4.91, 5.49] \text{ minutes} \end{aligned}$$

The PM's claimed value of 6 minutes lies **outside** this interval. This means the data is inconsistent with a true mean of 6 minutes at the 95% confidence level. We have strong evidence that the true mean is below 6 minutes — likely somewhere between 4.9 and 5.5 minutes.

This is also the setup for a formal hypothesis test (which we'll cover in Article 2) — but the confidence interval alone already gives a clear, interpretable answer that's easy to communicate to a non-technical PM.

Q9. What are the assumptions behind a classical confidence interval for the mean? What breaks if they're violated?

Answer:

Three main assumptions:

1. **Random sampling** — observations must be drawn randomly and independently from the population. If violated (e.g., you sampled only users from one city, or users in the same session influenced each other), the CI doesn't represent the population you're claiming to describe.
2. **Approximate normality of the sampling distribution** — guaranteed by CLT for large n (≥ 30 for moderate skew, more for heavy skew). For very small samples from skewed distributions, CLT hasn't kicked in and the z/t interval can be inaccurate. Bootstrap is a better option in that case.
3. **σ is estimated correctly** — the formula uses s as a stand-in for σ . This is fine for large n , but for small n it introduces extra uncertainty. The t -distribution (instead of z) accounts for this by using heavier tails.

If the random sampling assumption breaks, no amount of statistical machinery fixes it — the problem is in the design, not the analysis. If the normality assumption breaks, Bootstrap is the remedy.

Q10. Why is a 99% confidence interval wider than a 95% interval? Doesn't more data solve that?

Answer:

The width of a CI comes from two sources: the critical value (z^*) and the standard error ($s/\sqrt{n}$).

Going from 95% to 99% confidence increases z^* from 1.96 to 2.576 — a 31% increase — which directly widens the interval, *holding everything else constant*. This is unavoidable: to be more

certain you've captured the true value, you have to cast a wider net.

More data (larger n) reduces the standard error term ($s/\sqrt{n}$), which *narrows* the interval — but it doesn't change the z^* multiplier. So yes, you can compensate for a higher confidence level by also increasing n . In fact, the sample size formula does exactly this: if you want a 99% CI with the same width as a 95% CI, you can calculate the larger n needed to offset the bigger z^* .

The bottom line: confidence level and sample size are two independent controls on the CI. One widens it; the other narrows it. A good analyst uses both knobs deliberately.

Q11. What is the difference between a population parameter and a sample statistic? Give examples.

Answer:

A **population parameter** is a fixed (but usually unknown) numerical characteristic of the entire population. We denote these with Greek letters. Examples: μ (population mean), σ (population standard deviation), p (population proportion).

A **sample statistic** is a value calculated from observed data in your sample. It varies from sample to sample and is used to *estimate* the population parameter. We denote these with Latin letters. Examples: $\bar{x}$ (sample mean), s (sample standard deviation), $\hat{p}$ (sample proportion).

The fundamental goal of inferential statistics is to use sample statistics to make confident statements about population parameters — while honestly quantifying the uncertainty in that inference through tools like confidence intervals.

In practice: Spotify can never know the true proportion (p) of all 350M free users who would convert. But they can observe $\hat{p}$ from a sample of 10,000 and build a CI that gives an honest range for p .

Q12. Two data scientists build 95% CIs from the same population but different sample sizes — one uses $n = 100$, the other uses $n = 1,000$. How do the intervals compare?

Answer:

Both intervals are centered near the same value (both $\bar{x}$'s should be close to μ by CLT), but the one with $n = 1,000$ will be dramatically narrower.

The SE scales with $1/\sqrt{n}$:

- SE at $n = 100$: $s / \sqrt{100} = s / 10$
- SE at $n = 1,000$: $s / \sqrt{1,000} = s / 31.6$

The $n = 1,000$ interval has a standard error that's about $3.16\times$ smaller — so its margin of error is $3.16\times$ tighter. For example, if $s = 15$:

- $n = 100$: $\text{MoE} = 1.96 \times 1.5 = 2.94 \rightarrow \text{CI width of } \sim 5.9$

- $n = 1,000$: $\text{MoE} = 1.96 \times 0.47 = 0.93 \rightarrow \text{CI width of } \sim 1.9$

Both are valid 95% CIs — in repeated sampling, 95% of each type would contain μ . But the $n = 1,000$ interval is far more *useful* because its precision is much higher. This is why sample size planning matters: a valid CI that's 6 units wide may not be precise enough to drive a business decision.

Q13. Can you explain what "with replacement" means in Bootstrap sampling and why it matters?

Answer:

"With replacement" means that after you draw a data point from your sample to include in a bootstrap resample, you put it back before drawing the next one. So the same original data point can be selected zero times, once, or multiple times in a single bootstrap resample.

This is intentional and essential. The reason: Bootstrap is trying to simulate what would happen if you drew many different samples from the population. Since you can't actually re-sample from the population, you treat your original sample as a stand-in. Drawing *with* replacement from it is the mechanism that introduces the variability needed — each bootstrap resample is slightly different from the others, just like real samples from a population would differ.

If you sampled *without* replacement, every bootstrap resample would be identical to the original (just reordered), producing zero variability and a collapsed, useless CI. The "with replacement" is what makes Bootstrap work.

On average, each bootstrap resample will contain about 63.2% of the original data points (some are duplicated, some are missing) — this comes from the probability theory of sampling with replacement.

Q14. A product manager asks: "We ran a survey of 500 users and found 68% like the new feature. Can we say the majority of all users like it?" How do you respond?

Answer:

This is a great setup for building a proportion CI and letting it speak for itself.

$$\begin{aligned} n &= 500, \quad \hat{p} = 0.68 \\ \text{SE} &= \sqrt{(0.68 \times 0.32 / 500)} = \sqrt{0.000435} = 0.0209 \\ \text{MoE} &= 1.96 \times 0.0209 = 0.041 \\ \text{CI} &= [0.68 - 0.041, 0.68 + 0.041] = [0.639, 0.721] \\ &= [63.9\%, 72.1\%] \end{aligned}$$

Your response to the PM: "Yes, and we can quantify the certainty. With 95% confidence, the true proportion of all users who like the feature is between 63.9% and 72.1%. Since the entire interval

is well above 50%, we have strong statistical evidence that a majority of users (not just the sample) prefer the new feature."

The key discipline here is giving the PM not just the point estimate (68%) but the honest range — and then translating the range back into a business-actionable statement. This is what distinguishes a data scientist from someone who just reports numbers.

Q15. What are the practical differences between stratified sampling and simple random sampling? When would you choose one over the other?

Answer:

In **simple random sampling (SRS)**, every member of the population has an equal chance of selection — you draw blindly from the whole pool. It's easy to implement but can, by chance, under-represent important subgroups, especially small ones.

In **stratified sampling**, you divide the population into meaningful subgroups (strata) first, then sample from each stratum — usually proportionally (to mirror the population's composition) or equally (to ensure each group is represented enough for subgroup analysis).

Choose SRS when:

- The population is fairly homogeneous and subgroup analysis isn't needed
- Simplicity is valued and you have a large enough n that subgroups will naturally appear in proportion

Choose stratified sampling when:

- You need reliable estimates for specific subgroups (e.g., Swiggy wants delivery time estimates for Metro, Tier-2, and Tier-3 cities separately)
- Small but important subgroups might be missed by SRS (e.g., users on a rare device type)
- You want to reduce variance in your estimates — stratified sampling tends to produce more precise CIs than SRS for the same total n , because it removes between-stratum variability from the error

In a data science interview, the right answer is always: "it depends on whether subgroup accuracy matters and whether the population is heterogeneous." Recognising that sampling design is a deliberate choice — not just "grab some rows" — is what interviewers are listening for.
